



Amount of Data and Model Complexity

- One must match the *model complexity* to
 - **Data resources**
 - **Signal to noise ratio**
 - **Not target complexity**
- The rule of thumb is:

$$d_{VC}(\text{degrees of freedom of the model}) = N(\text{number of data points})/10$$

- In other words, 10 data points needed to fit a degree of freedom
- If the data is noisy, you need even more data

Overfitting as a Function of Data Resources, Model Complexity, Noise

- You can measure overfitting in terms of **generalization error** $\frac{E_{out} - E_{in}}{E_{out}}$
 - $\uparrow$ data resources $N \implies \downarrow$ overfitting
 - $\uparrow$ model complexity $d_{VC} \implies \uparrow$ overfitting
 - $\uparrow$ deterministic noise (target complexity) $\implies \uparrow$ overfitting
 - $\uparrow$ stochastic noise $\sigma^2 \implies \uparrow$ overfitting

- *Learning Curves*
- Learn-Validation Approach

Learning Curves vs Bias-Variance Curves

- **Learning curves** are the dual of the bias-variance curves
- For **bias-variance curves**

$$E_{in}, E_{out} = f(d_{VC}|N)$$

- Keep the complexity of training set fixed (number of examples N)
- Vary the model in terms of:
 - Model complexity d
 - Number of features p
 - Regularization amount λ
- For **learning curves**

Train/Test Split

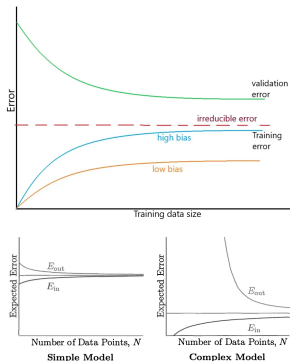
Train	Train	Train	Train	Test	Test
-------	-------	-------	-------	------	------

$$E_{in}, E_{out} = f(N|d_{VC})$$

- Fix the model
- Vary the size N of training set

Typical Form of Learning Curves

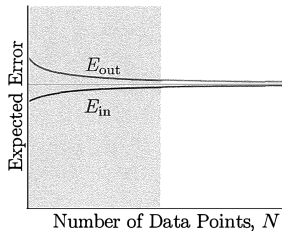
- Learning curves plot E_{in} and E_{out} as a function of data amount N , given the model $h()$
 - $E_{out} \geq E_{in}$ for any N
- Small N**
 - E_{in} is small (even 0)
 - The model is likely overfitted, memorizing and generalizing poorly
 - E_{out} is large
- Increasing N**
 - $E_{in} \uparrow$ as the model cannot fit all data
 - $E_{out} \downarrow$ as the model fits better and generalizes better
 - Generalization error $E_{out} - E_{in} \downarrow$
- $N \rightarrow \infty$
 - E_{in} reaches a maximum (irreducible error)
 - E_{out} reaches a minimum
 - $E_{out} - E_{in}$ depends on the complexity of the model



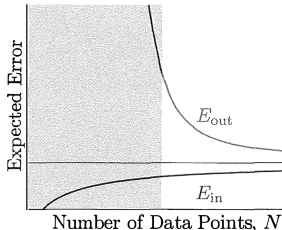
High-Bias vs High-Variance Regime

- From the learning curve you can see the two regimes:
 - High-variance regime** for small N
 - E_{in} is small
 - Small data set $D \implies$ high dependency on D
 - More data helps reduce gap between E_{in} and E_{out}
 - High-bias regime** for large N
 - E_{in} is large; flattens for large N
 - Best model fitted; more data won't help
 - E_{out} can be close to E_{in} (good generalization) or not

Learning curves for $\mathcal{H}_2$



Learning curves for $\mathcal{H}_{10}$



- Learning Curves
- ***Learn-Validation Approach***
 - Train / Test
 - Cross-Validation

- Learning Curves
- Learn-Validation Approach
 - *Train / Test*
 - Cross-Validation

Estimating Out-Of-Sample Error with One Point

- Pick a **single out-of-sample point** $(\underline{x}', y)$
- The error of the model h is:

$$E_{val}(h) = e(h(\underline{x}'), y)$$

where the error can be:

- Squared error $(h(\underline{x}) - y)^2$
- Binary error $I[h(\underline{x}) - y]$
- ...
- The **error on an out-of-sample point** is an **unbiased estimate** of E_{out} , since:
$$\mathbb{E}[E_{val}(h)] = \mathbb{E}[e(h(\underline{x}), y)] = E_{out}$$
- The quality of the estimate depends on the standard error $\mathbb{V}[e(h(\underline{x}), y)]$, which in an unknown value

Estimating Out-Of-Sample Error with K Points

- To improve the estimate, use a validation set with K points $(\underline{x}_1, y_1), \dots, (\underline{x}_K, y_K)$ drawn IID
- Compute the **error on the validation set** as:

$$E_{val}(h) = \frac{1}{K} \sum_{i=1}^K e(h(\underline{x}_i), y_i)$$

- The validation error is an **unbiased estimate** of out-of-sample error since:

$$\mathbb{E}[E_{val}(h)] = E_{out}(h)$$

- The standard error is $\sqrt{K}$ smaller:

$$\mathbb{V}[E_{val}(h)] = \frac{1}{K^2} \sum_i \mathbb{V}[e(h(\underline{x}_i), y_i)] + \text{covariances}$$

(covariances are 0 because $\underline{x}_i$ independent)

$$= \frac{1}{K^2} \sum_i \mathbb{V}[e(h(\underline{x}_i), y_i)]$$

$$= \frac{K\sigma^2}{K^2} = \frac{\sigma^2}{K}$$

Trade-Off Between Training and Validation Set

- **Problem:** you have finite amount of data points N

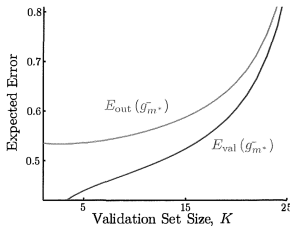
$$D_{val} = \{K \text{ points}\}$$

$$D_{train} = \{N - K \text{ points}\}$$

- You know that:

$\uparrow K \implies |E_{val} - E_{out}| \downarrow$ (most reliable estimate)

$\implies \downarrow N - K \implies E_{in}, E_{out} \uparrow$ (worse model)



- If K is too big, you get a **reliable estimate of a bad number!**
- **Solution:**
 - Rule of thumb: 70-30 or 80-20 split between train and validation

Error From VC Analysis vs Learn-Validation Approach

- In general:

$$E_{out}(h) = E_{in}(h) + \text{generalization error}$$

- **VC analysis**

- Estimates generalization error as “overfit penalty” in terms of hypothesis set complexity

- **Learn-validation**

- Estimates E_{out} directly by holding out data as validation set:

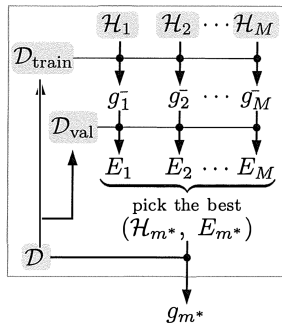
$$E_{val} \approx E_{out}$$

- Use learn-validation approach at different points of the modeling flow:
 - For selecting hyperparameters (validation set)
 - To estimate final performance (test set)

Reusing Validation / Test Set for Training

- Any data used for learning (e.g., model training or model selection)
 - Is biased and optimistic
 - Cannot be used to assess the learned model
- Never use** D_{val} / D_{test} for training or D_{train} for evaluation
- After research is done, all available data can be used to learn
- Algorithm**

- Train with $N - K$ points to learn g^-
- Use K points to compute $E_{val}[g^-]$ estimating $E_{out}[g^-]$
- Once the model form is finalized, use all N data points (including validation, test set) to re-train to get g
 - $E_{val}[g] < E_{val}[g^-]$ since g is learned on a larger data set than g^- and is thus better
- Deliver to customers:
 - Final hypothesis g
 - Upper bound of out-of-sample performance $E_{val}[g^-]$



Learn-Validation Approach: Pros and Cons

- **Pros**
 - Estimate E_{out} using E_{val}
 - Simple to compute, no complexity from VC analysis
- **Cons**
 - Cannot use all data for learning and validation; need a compromise
 - Learned model and E_{val} depend on the split; different splits can give different results

- Learning Curves
- Learn-Validation Approach
 - Train / Test
 - ***Cross-Validation***

Cross-validation

- **Divide dataset** into K folds, each with $\frac{N}{K}$ samples
 - Each fold should reflect dataset statistics
 - E.g., stratified sampling
- **Do K iterations** $i = 1, \dots, K$
- In i -th iteration
 - Train on all folds except i using $\frac{K-1}{K}N$ points and get $g^{(-i)}(\underline{x})$
 - Validate on i -th fold using $\frac{N}{K}$ points to compute

$$E_{val}^{(i)} = E_{val}[g^{(-i)}(\underline{x})]$$

- **Average** K error rates and compute bounds:

$$E_{val} = \frac{1}{K} \sum_i E_{val}^{(i)}$$

Train / test validation

Iteration 1	Test	Train	Train	Train	Train
Iteration 2	Train	Test	Train	Train	Train
Iteration 3	Train	Train	Test	Train	Train
Iteration 4	Train	Train	Train	Test	Train
Iteration 5	Train	Train	Train	Train	Test

5x cross-validation

Iteration 1	Test	Train	Train	Train	Train
Iteration 2	Train	Test	Train	Train	Train
Iteration 3	Train	Train	Test	Train	Train
Iteration 4	Train	Train	Train	Test	Train
Iteration 5	Train	Train	Train	Train	Test

Cross-Validation: Pros and Cons

- **Pros**

- Efficient data usage
 - All data used for learning and validation
- Better estimate of E_{val} than separate training/validation sets
- Folds can be stratified

- **Cons**

- Computationally intensive: K learning phases
- Dependency on fold selection
- Errors E_{val} are not independent
 - Coupling through common training samples
 - Experimentally not completely correlated

Repeated Cross-Validation

- **Repeated Cross-Validation**
 - Cross-validation results depend on fold selection
 - To remove this dependency, repeat cross-validation multiple times (e.g., 10) and average results
 - Note that “*10x 10-fold cross-validation*” is different than “*1x 100-fold cross-validation*”

Leave-One-Out Cross-Validation

- **Leave-one-out** (LOO) cross-validation
 - There are N training sessions
 - Each session trains on $N - 1$ points and validates on 1 point
 - Equivalent to “ N -fold cross-validation” where N is the number of examples in the dataset
- **Train**
 - For i -th fold, $N - 1$ samples for training $\implies g^{(-i)} \approx g$
- **Validate**
 - Estimate the validation error on a single point (bad):

$$E_{val}[g^{(-i)}] = e(g^{(-i)}(\underline{x}_i), y_i) \not\approx E_{out}[g^{(-i)}]$$

- Average E_{val} over the points as estimate of E_{out} (good):

$$E_{val} = \frac{1}{N} \sum_{i=1}^K E_{val}[g^{(-i)}]$$

Leave-One-Out Cross-Validation: Pros and Cons

- **Pros**

- Max data used for training
- Deterministic procedure
 - No dependency on fold selection

- **Cons**

- High computational cost (as many learning phases as data points)
- Cannot be stratified
- Higher correlation between cross-validation estimates

Bootstrap

- **Algorithm**
 - Pick N samples with replacement from a data set with N instances to build training set
 - Pick the elements never chosen to build the test set (“out-of-bag” samples)
 - Training set contains 63.2% of all the samples, 36.8% in the test set
- **Pros**
 - Works for small data sets since it “expands” the data
- **Cons**
 - Not flexible
 - Smaller percentage of instances are used for training set than 10-fold cross validation

Bootstrap: Problem

- Given $X \sim F$
 - Draw n IID samples X_i from F
- **Goal:**
 - Consider a sampling statistic $T = g(X_1, \dots, X_n)$
 - E.g., mean, median, OLS coefficients, ...
 - You want to:
 - Estimate distribution of T , e.g., $F_T(x), f_T(x)$
 - Estimate a statistic of T (e.g., mean, std err)
 - Constructing confidence intervals: e.g., $\mu \pm \epsilon$
 - Calculating standard errors: e.g., $\sigma(\hat{T}_n)$
 - Hypothesis testing

Bootstrap Procedure: Algorithm

- Use observed data $X_1, \dots, X_n$ to construct estimated population distribution $\hat{F}_n$
- Repeat B times:
 - Draw n samples with replacement from estimated population distribution $\hat{F}_n$
 - Sampling with replacement from X_i equals sampling from $\hat{F}_n$
 - Compute sample statistics from n samples: $T^{(i)} = g(X_1^{(i)}, \dots, X_n^{(i)})$
- Use B samples of $T^{(i)}$ to estimate empirical distribution $\hat{T}$
- Compute statistics (e.g., confidence interval, standard error) of T from empirical distribution of $\hat{T}$
- Conceptually
 1. Pretend empirical distribution is real
 2. Sample with replacement

Bootstrap: Pros

- Tremendously useful tool
- Fewer assumptions
 - No need for simplifying assumptions for closed formulas
 - Data need not be Gaussian
- Greater accuracy
 - Does not rely on large sample sizes, unlike Central Limit Theorem (CLT) / Law Large Numbers (LLN)
- Generality
 - Applies to any sample statistics, even non-linear ones (e.g., median)
- Simulation vs math
 - Bootstrap frees data scientists from complex math, approximations, and asymptotics
- Bootstrap is a non-parametric method
 - Parametric method assumes a model and estimates parameters
 - Bootstrap makes no distribution assumptions

Bootstrap: Example of Die Rolls

- Compute distribution of sum of rolling a die 50 times

$$Y = \sum_{i=1}^{50} X_i = g(X_1, \dots, X_{50})$$

- Sample statistics similar to sample mean, but can be anything
- **Solution**
 1. By math
 - If you know PMF of the die, compute distribution using mathematics
 - Theorem of lazy statistician for mean, variance
 - Compute PDF of Y by convolving PDFs
 2. By sampling (real or simulated)
 - Roll die 50 times
 - Compute sample statistic
 - Repeat procedure
 - Plot approximate distribution of Y
 3. If only 50 samples of die are known $\rightarrow$ bootstrap

Bootstrap of the Median: Pseudo-Code

```
def bootstrap_median(x, n_boot):  
    # Compute n_boot sample statistics.  
    median_boot = [0.0] * n_boot  
    for i in range(n_boot):  
        # Sample with replacement.  
        x_star = sample with replacements from x  
        # Compute median for bootstrapped samples.  
        median_boot[i] = median(x_star)  
    # Compute mean and std err from approximation of sample statistics.  
    m_median = numpy.mean(median_boot)  
    se_median = numpy.std(median_boot)  
    return m_median, se_median
```

Bootstrap for variance of sample statistics: explanation

- Under the hypotheses of bootstrap:
 - $X \sim F$
 - n samples X_i IID from F
 - Statistic $T = g(X_1, \dots, X_n)$
 - Compute $\mathbb{V}_F[T]$ (variance of sample statistics), where F is unknown
- **First approximation**
 - Only samples from F available
 - Approximate F with $\hat{F}$, as empirical CDF $\hat{F}$ converges to F

$$\mathbb{V}_F[T] \approx \mathbb{V}_{\hat{F}}[T]$$

- Approximation
 - Not small
 - Depends on sample size and shape of F
- **Second approximation**
 - Exact $\hat{F}$ may lack closed formula for $\mathbb{V}_{\hat{F}}[T]$
 - Use LLN and simulation to approximate variance:

$$\mathbb{V}_{\hat{F}}[T] \approx v_{boot} = \frac{1}{B} \sum_i (T_i - \bar{T})^2$$

Approximation size reduced by increasing B